

The Compose Logbook — Weekly Ship Report

One page, every class. Sketch it, ship it, log the diff — watch your app grow week over week.

NAME

you — the builder

DATE

today, YYYY-MM-DD

CLASS

which class this covers, e.g. 08

APP NAME

your semester app + 3–5 words on what it does

THIS WEEK'S THEME

ONE phrase, e.g. "wire the data layer"

BUILD

version bump so progress shows: v0.6 → v0.7

REPO / COMMIT

link this week's commit or PR — the proof behind your Diff

ENERGY

one emoji for your build mood

► How to use this each class

- Duplicate LAST week's filled-in copy and rename it (e.g. "Logbook — Class 08"). You fill this by DIFFING against last week, so keep the old one open beside it.
- Fill top-to-bottom in 10–15 minutes. Phrases, checkmarks, and a sketch — not paragraphs. If a section takes more than ~2 minutes, write less.
- Each section has ONE job, so you only type things once: the Diff is WHAT changed · the Design Log is WHY · the Plan is NEXT. Carried-over items copy verbatim — don't reword them.
- Paste a real screenshot / emulator capture where you can; a quick boxed wireframe is fine otherwise. Always note light/dark and which UI state is showing.
- First class, or missed a week? Skip Burndown and the Diff's ~/- lines — list what exists now as all "+" lines and just fill the Plan. You'll diff for real next week.
- Write in Compose vocabulary on purpose (NavKey, ViewModel, StateFlow, sealed UI state, modifier order) — filling this page is your weekly review of the course.

Burndown — last class's promises

Read back what you committed to at the END of last class and mark what actually happened. Closes the loop so “progress” is verifiable, not just claimed.

- Paste each commitment from last week's “Plan for next class” into a row, then mark it Done / Partial / Carried / Dropped.
- If it isn't Done, add a one-phrase why. Carried rows copy VERBATIM into this week's Plan below — don't reword them.

EXAMPLE (1) Add RideDetailScreen via Nav3 — Done. (2) Wire RideRepository to emit a sealed RideUiState — Partial (Error branch still a TODO). (3) Add a “seats left” Slider — Carried (ran out of time after the nav refactor).

Committed last class	Status (Done / Partial / Carried / Dropped)	Why (one phrase, if not Done)
e.g. Add RideDetailScreen via Nav3	Done	—
e.g. Wire the data layer to a sealed UiState	Partial	Error branch still a TODO

The Diff — what shipped since last class

One headline plus the delta only, in git-diff style. The markers structurally stop you from re-listing old work.

- Top line — “Shipped this week”: in ONE sentence, what a classmate reopening your app would notice is new.
- Then 3–6 delta lines, each prefixed: + added · ~ changed existing · – removed/redid (just like a git diff).
- The WHAT only — save every “why” for the Design Log so you don't type it twice.

EXAMPLE Shipped: the ride-detail screen is live. + Added RideDetailScreen (NavKey RideDetail(rideld)). + Added RideRepository emitting a sealed RideUiState. ~ RideListViewModel now exposes StateFlow<RideUiState> instead of a raw List. – Removed the hard-coded sampleRides list from the Composable.



Design decision log

The design steps and the WHY behind your choices — the trade-offs you made on purpose. At least one UI/layout decision AND one architecture/state decision each week.

- Log each real decision. The Why must name a concrete force — rotation, screen size, scannability, testability, back-stack depth, recomposition cost. "It's cleaner" is not a Why.
- If you can't name what you gave up in the Trade-off column, you didn't make a decision — you just coded. Write "see Design Log" elsewhere instead of repeating it.

EXAMPLE UI — Gave *RideDetail* its own *Nav3* screen | Why: a map + notes + driver card is too much for a list row | Rejected: an inline-expanding Card (it pushed siblings off-screen in the Playground) | Trade-off: one extra back-stack entry, but the list stays scannable.

Decision (what I chose)	Why (a concrete force)	Alternative considered	Trade-off / what I gave up
e.g. <i>RideDetail</i> gets its own <i>Nav3</i> screen	Too much content for a list row	Inline-expanding Card	+1 back-stack entry; list stays scannable
e.g. Hoist search text into the <i>ViewModel</i>	Survives rotation; <i>TextField</i> stays previewable	<i>remember{} in the Composable</i>	A little more wiring now

Architecture & state (in Compose terms)

Snapshot your app's plumbing in the course's vocabulary. Doubles as a weekly self-test.

- Fill the table: how did Navigation, State, and the Data layer change this week — and what's still TODO? If a row just repeats a Design Log decision, write “see Design Log”.
- Then mark your main screen's sealed UI-state branches below: which are REAL vs still faked.

EXAMPLE *Navigation: before [RideList] → after [RideList, RideDetail(id)]; TODO: pass rideId into the detail ViewModel cleanly. State: raw List → StateFlow<RideUiState> via collectAsStateWithLifecycle(). Data: hard-coded list → RideRepository emitting the sealed state; TODO: a real source.*

Layer	Before	After	Still TODO
e.g. Navigation (Nav3 keys / back stack)	[RideList]	[RideList, RideDetail(id)]	pass rideId to the ViewModel
e.g. State (ViewModel / StateFlow)	raw List in the Composable	StateFlow<RideUiState>	hoist the seat filter
e.g. Data (repository)	hard-coded list	RideRepository → sealed state	a real network/db source

My main screen's sealed UI state — mark each branch (✅ real · ❌ still TODO):

- | | |
|--|--|
| ☐ Loading — real / TODO | ☐ Error — real / TODO |
| ☐ Empty — real / TODO | ☐ Success — real / TODO |

UI this week — sketch + the pieces behind it

Show the actual UI that changed plus the exact Compose building blocks behind it. Reinforces “order matters” and the light/dark @Preview habit.

- Paste a screenshot / emulator capture OR sketch a boxed wireframe of the screen(s) you touched. Label the containers (Column / Row / Box / LazyColumn / Card / Scaffold) and caption it: screen name + UI state shown + light/dark.
- Write ONE modifier chain IN ORDER and a one-line note on why the order matters (size-affecting vs draw-only) — or what breaks if you reorder it.
- Then tick the Material 3 pieces you used this week.

 Sketch your screen here — or paste a screenshot / @Preview render

Material 3 pieces I used this week (★ anything used for the first time):

- | | |
|---|---|
| ☐ Column / Row / Box | ☐ TextField / OutlinedTextField |
| ☐ LazyColumn | ☐ Switch / Checkbox / Slider |
| ☐ Card / Surface | ☐ Scaffold / top bar |
| ☐ Button family (Filled / Tonal / Outlined / Text) | ☐ Modifier order mattered somewhere |
| | ☐ Verified in BOTH light and dark @Preview |

✓ What works right now — the demo

Prove the build runs: the 30-second demo you could give today, with an honesty flag so a slick screenshot can't oversell a stub.

- Write the tap-path you'd demo live — what you can do this week that you couldn't last week.
- Disclose what's faked / stubbed / hard-coded behind the demo, and where it runs (device/emulator vs @Preview only).

EXAMPLE *Demo: launch → RideList shows 3 rides → tap one → Nav3 pushes RideDetail(id) → ~0.5s spinner → Success renders route + seats → Back returns with scroll kept. Runs on the Pixel 6 emulator. Faked: the 3 rides are hard-coded in FakeRideRepository.*

Green checks — tick only what genuinely passes right now:

- | | |
|---|--|
| ☐ Builds & runs on emulator/device with no crash | ☐ Light & dark both look right |
| ☐ The new feature is tap-through demoable | ☐ No obvious jank / overflow |
| | ☐ Pushed — commit link pasted in the header above |

Blockers & open questions

Surface what's stuck and the exact questions you want answered — specific enough to act on in class. Triage, not failure.

- List 1–3 blockers. Name the Composable / ViewModel / concept, rate severity, say whether you're Blocked or just Slowed, and what you've already tried.
- Star ★ the ONE question most worth class time.

EXAMPLE *Blocked (High): can't pass rideId into RideDetailViewModel cleanly — through the NavKey or a SavedStateHandle? Tried reading it off the back-stack key directly, felt wrong. ★ Top question: what's the idiomatic way to get a NavKey's argument into a ViewModel?*

Blocker (name the Composable / ViewModel / concept)	Severity (Low / Med / High)	Blocked vs Slowed	What I've already tried
<i>e.g. Get rideId into RideDetailViewModel</i>	<i>High</i>	<i>Blocked</i>	<i>read it off the back-stack key — felt wrong</i>

Triage signal:

☐ Stuck > 30 min — I want you to look at this with me in class

Plan for next class — commitments

Pre-write next week's work as small, verifiable cards so you start from momentum, not a blank page. Carried items from Burndown flow in here.

- Commit to 1–3 small, demoable cards. Pull in anything marked Carried above (verbatim).
- Write a one-line done-test for the ★ MUST-DO: “done = I can tap ____ and see ____ in the Success state.”
- Note what you'll prototype in the Compose Playground first, and draft next week's commit headline now (imperative).

EXAMPLE ★ *MUST-DO: wire the Error branch + a Retry Button (done = kill the network, see the error UI, tap Retry, see Success). Stretch: add the seats-left Slider [carried]. Playground first: the request-form layout. Commit headline: “feat: add error+retry to RideList”.*

☐ ★ MUST-DO card — the one thing that, if only one ships, ships

☐ ★ MUST-DO done-test: done = I can tap ____ and see ____

☐ Stretch card 2 (mark [carried] if from Burndown)

☐ Stretch card 3

☐ Prototype in the Compose Playground first →

☐ Draft next week's commit headline (imperative) →

Confidence dials (1–5)

A fast, honest self-rating to track growth over the semester and surface what to fix next. Just tick — then name your lowest axis as next week's focus.

- Mark one box per row (1 = shaky · 5 = could teach it).
- Your lowest axis is next week's focus — circle it.

Area (1 = lost · 5 = confident)	1	2	3	4	5
Progress this week	☐	☐	☐	☐	☐
I understand my own code	☐	☐	☐	☐	☐
Compose & Material 3 fluency	☐	☐	☐	☐	☐
Architecture (Nav3 + ViewModel/StateFlow + data layer)	☐	☐	☐	☐	☐
Energy / momentum	☐	☐	☐	☐	☐

Brag, bug & codename (the fun close)

End on a high note so the page stays human and motivating to read back later.

-  Brag: one moment you felt like a real Android dev this week.
-  Bug of the week: the funniest/dumbest bug you hit and the fix — one line.
-  Learned +  Codename: one post-it-worthy line of Compose wisdom, then a fun release codename for this build.

EXAMPLE  Brag: the list updated itself after the repo emitted a new state — zero refresh code, declarative UI finally clicked.  Bug: blank screen for 20 min — my Column wasn't scrollable; swapped to LazyColumn.  Learned: clip the shape BEFORE you paint the background.  v0.7 — "Carpool Lane".

Small commits, every class. Duplicate this page next week and diff against today — that growing stack of logbooks IS your app shipping. 🚀